

**CURSO PROFISSIONAL TÉCNICO DE GESTÃO E
PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS**

PROVA DE APTIDÃO PROFISSIONAL



imoral

Nome do aluno: Arthur de Mello Mattos Dionisio Amaral

Diretora de Curso: Maria Luísa Parente

Orientador do Projeto: Professor José Augusto da Rocha Gonçalves

Ano e Turma: 12º TGPSI

Ciclo 2023-2026

1. Agradecimentos

Queria dedicar alguns agradecimentos a pessoas que me ajudaram durante o percurso do desenvolvimento do projeto.

Primeiro queria agradecer a Fábio Vitoriano parceiro de projeto e foi quem teve a ideia inicial do projeto no nosso 10º ano sem ele o projeto seria impossível de se realizar.

Logo em seguida nosso orientador de projeto José Augusto da Rocha Gonçalves, apoiando-nos em cada etapa e se preocupando a cada estagio muito mais do que o necessário, também sem ele nunca conseguiria ter o projeto na fase atual.

Também queria agradecer a nossa diretora de curso Maria Luísa Parente foi nas aulas dela em que tivemos a ideia do projeto e que construímos a base dele.

Queria agradecer minha mãe Carla Mattos que sempre me deu ideias e me influenciou a ser mais criativo e uma pessoa que tem vontade de criar e se expressar.

Além disso também agradeço todos os amigos, familiares, professores e conhecidos que me apoiaram e incentivaram a ideia do projeto e me ajudaram a seguir até o fim com os objetivos que pretendia.

2. Índice

1. Agradecimentos.....	2
2. Índice.....	3
3. Resumo.....	5
5. Introdução	6
5.1 Contexto	6
5.2 Motivação	6
5.3 Objetivos	7
5.4 Estrutura do Relatório	9
6. Análise de Requisitos	10
6.1 Requisitos Funcionais	10
6.2 Requisitos Não Funcionais	12
6.3 Restrições e Limitações	13
7. Metodologia	14
7.1 Metodologia de Desenvolvimento	14
7.2 Ferramentas e Tecnologias Utilizadas	15
7.3 Cronograma	17
8. Desenvolvimento	20
8.1 Arquitetura do Sistema	20
8.2 Design da Interface	24

8.3 Implementação.....	27
8.4 Testes e Validação.....	36
9. Resultados.....	40
9.1 Funcionalidades Desenvolvidas.....	40
9.2 Benefícios Esperados.....	41
9.3 Limitações.....	41
10. Conclusão.....	42
10.1 Síntese do Projeto	42
10.2 Reflexão Crítica.....	42
10.3 Melhorias Futuras.....	43
11. Referências.....	44

3. Resumo

“imoral”, essencialmente, é um projeto de marca de roupa com inspirações de estilo alternativo que para além da venda e personalização de roupas tenta também criar um espaço de comunidade. Mas não é apenas uma empresa ou marca de roupa é um movimento contra a padronização, e à favor do crescimento e desenvolvimento de outras marcas independentes e onde se partilha ideias criativas e inovadoras de expressão pessoal. O público-alvo do projeto são pessoas que querem desenvolver seu estilo, pessoas alternativas e marcas independentes de Portugal.

Neste projeto foram utilizadas algumas linguagens para a desenvolver uma App Android, um site promocional, um Painel de Administração, base de dados e API, e essas foram: Java, HTML, CSS, JavaScript. PHP, SQL.

O que pretendo ter com este projeto é ter uma App com uma loja com compras funcionais e um espaço comunidade com rede social, um Painel de Administração que tenha tudo necessário para gerir o negócio, tudo conectado com uma base de dados bem estruturada e API.

O estado atual do projeto é um sistema local funcional, app e o site estão integrados com a base de dados e API, é possível realizar operações básicas e perto de concluir a simulação de compra na App. O espaço comunidade já mostra os posts da base de dados, mas utilizadores ainda não conseguem fazer posts.

4. Palavras Chave

Movimento contra padronização; Expressão pessoal; Desenvolvimento de Aplicação Android; Base de Dados; Comunidade Online; Marcas Independentes.

5. Introdução

5.1 Contexto

Nos dias atuais verifica-se que cada vez mais temos menos a vontade de expressão pessoal e isso é principalmente culpa do contexto em que vivemos. Falando especificamente da expressão pessoal em forma da vestimenta, a indústria faz-nos acreditar que roupa é só uma estética e aparência sendo que é das principais formas de falarmos e passarmos quem somos. Para a indústria é mais lucrativo, quando as pessoas não se preocupam em se expressar as pessoas compram as mesmas coisas e a indústria fast-fashion pode continuar com o seu método de produzir quantidades enormes de roupas iguais de qualidade baixa.

Com isso pessoas alternativas que ainda têm essa vontade de expressão criam marcas independentes, mas com as condições atuais elas não conseguem se manter por muito tempo e é muito difícil delas realmente se consolidarem e se tornarem relevantes.

Assim surgiu a ideia da "imoral". Consiste em uma loja de roupa que também inclui um espaço comunidade/rede social que quer juntar essas marcas independentes para crescerem juntas como um coletivo. Pensando no cenário português de marcas independentes verifica-se que elas já têm uma tendência a se juntar, mas não têm uma maneira fácil de fazê-lo para se manter então a imoral seria ideal.

5.2 Motivação

A motivação deste projeto surge do meu interesse pessoal e curiosidade por costura e design de roupas e acessórios e da falta de lojas independentes com roupas de estilos que muitos jovens se interessam e não encontram facilmente, também por essa razão, tenho vontade de desenvolver um centro de comunidade em que possamos partilhar e divulgar outras lojas independentes, contribuindo o crescimento de lojas como essa que desempenham um papel importante dentro das comunidades alternativas.

Além disso também vejo esse projeto como uma forma de testar e melhorar as minhas capacidades em fotografia, design e produção visual que são outras áreas que têm grande impacto na minha vida.

Então aplicando os conhecimentos dados nas aulas foi possível obter as bases necessárias para tornar esta ideia realidade.

5.3 Objetivos

Objetivos Gerais:

Objetivos gerais deste projeto são principalmente criar uma loja de roupas e acessórios customizados para apresentarmos a cultura alternativa para outras pessoas, e ainda, facilitar para pessoas já dentro dessa comunidade desenvolverem o seu estilo próprio. Segunda parte do projeto seria criar um espaço comunidade para divulgação, partilha, e troca de ideias sobre outras marcas de roupa, estilos ou até sugestões/opiniões sobre essas marcas independentes.

Objetivos Específicos:

Utilizador

- Registo e Login;
- Perfis de utilizador com Nome, avatar, bio;
- Possibilidade publicações/comentários na comunidade.

Loja

- Encomendas;
- Catálogo;
- Carrinho;
- Wishlist;
- Avaliações/Reviews.

Personalização

- Aba para personalização;
- Editor visual simples (com imagem pré-feitas) - Preço seria dinâmico conforme personalização;

Espaço comunidade

- Publicar posts (texto, imagem, link);
- Curtir e comentar;
- Seguir outros utilizadores/marcas;
- Categorias de posts (ex: “Novas marcas”, “Ideias de design”, “Inspiração visual”);
- Hashtags ou tags temáticas.

Gerais

- Site para desktop e telemóvel;
- Aplicação para telemóvel.

Painel Administrativo

- Gestão de produtos, categorias e pedidos;
- Moderação da comunidade (posts e comentários);
- Estatísticas de vendas.

5.4 Estrutura do Relatório

O Relatório está organizado em capítulos esses são: o Índice mostrando as páginas em que cada tópico é desenvolvido, os Agradecimentos onde são mencionadas pessoas importantes para o desenvolvimento e a conclusão do projeto, o Resumo onde é resumido o que é e o que foi desenvolvido no projeto, as Palavras Chave, a Introdução onde é apresentado o contexto a motivação e os objetivos do projeto, a Análise de Requisitos tudo o que queríamos com o projeto, a Metodologia qual metodologia utilizamos para desenvolver o projeto, o Desenvolvimento o que foi desenvolvido com o projeto, os Resultados os resultados que obtivemos, a Conclusão que apresenta uma síntese e diz sobre o futuro do projeto e Referências as referências utilizadas.

6. Análise de Requisitos

6.1 Requisitos Funcionais

Utilizador

- Registo e Login. O sistema deve permitir que utilizadores façam registo e login com suas credenciais.
- Perfis de utilizador com Nome, avatar, bio. O sistema deve permitir que os utilizadores criem perfis e personalizem suas informações.
- Possibilidade publicações/comentários na comunidade. O sistema deve permitir que os utilizadores interajam com publicações e comentários no espaço comunidade.

Loja

- Encomendas. O sistema deve permitir que os utilizadores realizem encomendas dos produtos disponíveis.
- Catálogo. A aplicação deve mostrar um catálogo com todos os produtos disponíveis para compra.
- Carrinho. O sistema deve ter um carrinho funcional para utilizadores conseguirem fazer compras.
- Wishlist. O sistema deve ter uma wishlist para utilizadores possam demonstrar e guardar produtos que tenham interesse.
- Avaliações/Reviews. O sistema deve permitir que utilizadores façam avaliações dos produtos.

Personalização

- Aba para personalização. O sistema deve ter uma aba para personalização onde utilizadores possam fazer produtos mais únicos e como desejam.
- Editor visual simples (com imagem pré-feitas) - Preço seria dinâmico conforme personalização. A aplicação deve ter um editor visual simples para tornar a personalização mais intuitiva e simples.

Espaço comunidade

- Publicar posts (texto, imagem, link). A aplicação deve permitir que seja possível fazer posts com texto, imagens e links.
- Curtir e comentar. A aplicação deve permitir que os utilizadores curtam e comentem em posts ou comentários.
- Seguir outros utilizadores/marcas. O sistema deve permitir que utilizadores sigam ou sejam seguidos por outros utilizadores ou marcas.
- Categorias de posts (ex: “Novas marcas”, “Ideias de design”, “Inspiração visual”). O sistema deve permitir que posts possam ter categorias ou temas.
- Hashtags ou tags temáticas. O sistema deve permitir que existam hashtags ou tags para assuntos ou temas específicos.

Gerais

- Site para desktop e telemóvel. O sistema deve ter um site para desktop e telemóvel.
- Aplicação para telemóvel. O sistema deve ter uma aplicação em Android para telemóvel.

Painel Administrativo

- Gestão de produtos, categorias e pedidos. O painel administrativo deve permitir fazer a gestão de produtos, categorias e pedidos.
- Moderação da comunidade (posts e comentários). O painel administrativo deve permitir fazer a moderação da comunidade como eliminar posts e comentários e ativar, suspender ou banir utilizadores.
- Estatísticas de vendas. O painel administrativo deve permitir ver as estatísticas e dados de todas as vendas da loja.

6.2 Requisitos Não Funcionais

Desempenho

- Tempo de resposta da API e da aplicação. O sistema deve fazer com que o tempo de resposta entre a API e aplicação seja o mais breve possível.
- A app deve carregar de forma rápida e fluida. O sistema deve permitir que a app carregue de forma rápida e fluida.

Segurança

- Passwords devem ser encriptadas. O sistema deve garantir que as palavras-passe dos utilizadores sejam encriptadas e seguras.
- Autenticação de utilizadores protegida. O sistema deve garantir que a autenticação dos utilizadores seja segura.
- Moderação para proteger a comunidade. O sistema deve garantir que haja a moderação necessária para proteger e manter a comunidade um ambiente seguro.

Usabilidade

- Interface intuitiva e fácil de usar. A app e site devem ter uma interface intuitiva e fácil de usar para que o tempo de adaptação à aplicação seja reduzido.
- Design adaptado ao público-alvo jovem e alternativo. A app e site devem ter um design adaptado ao público-alvo do projeto tendo em conta o contexto em que estamos.

Compatibilidade

- App funciona em Android. A app deve funcionar em Android.
- Site funciona em desktop e telemóvel. O site deve ter compatibilidade para desktop e telemóvel.

6.3 Restrições e Limitações

O sistema não permite publicar mais que uma foto por publicação. A função de loja e espaço comunidade são apenas disponíveis para App Android. Não é possível selecionar produtos específicos dentro do carrinho para compra. Compras reais ainda não podem ser efetuadas.

7. Metodologia

7.1 Metodologia de Desenvolvimento

O projeto foi realizado em dupla, por isso dividimos as tarefas, revemos e testamos mutuamente as partes desenvolvidas por cada um, garantindo que todas as etapas estão corretas e a funcionar como previsto.

A nossa forma de trabalho baseia-se na Metodologia Ágil de Desenvolvimento. Isto significa que seguimos um ciclo composto pelos seguintes passos:

- Planear as tarefas a realizar;
- Criar layouts ou protótipos para consolidar as ideias visuais e funcionais do software;
- Desenvolver funcionalidades necessárias;
- Testar o que foi programado;

Após a validação, reiniciar o ciclo, aperfeiçoando continuamente o produto.

Ao utilizar esta metodologia, conseguimos manter um projeto flexível e adaptável a qualquer necessidade ou imprevisto, permitindo resolver problemas com maior facilidade e garantir que o software evolui de forma consistente e eficiente.

A gestão e partilha do código foi realizada com o GitHub, permitindo um controlo de versões ao longo do desenvolvimento.

7.2 Ferramentas e Tecnologias Utilizadas

Linguagens de Programação

Java e XML -

Java, foi utilizado para o desenvolvimento da parte lógica da aplicação Android, incluindo a comunicação com a API e as funcionalidades da app. Enquanto o XML foi utilizado para o desenvolvimento da parte visual da aplicação.

HTML, CSS, JavaScript -

HTML, foi utilizado para o desenvolvimento da base do site promocional e o painel de administração. CSS, serviu para estilizar e definir modelos visuais em partes do painel de administração. O JavaScript foi utilizado para gerir a parte lógica de ambos os sites como os gráficos e operações de comunicação com a base de dados.

PHP -

PHP, foi utilizado para o desenvolvimento da comunicação com a base de dados e API. A comunicação entre a API e a aplicação é feita através do formato JSON.

SQL -

SQL, foi utilizado para desenvolver toda a parte da base de dados com MySQL, sendo essas a estrutura da base de dados em si e as queries.

Ambientes de Desenvolvimento - Android Studio, VS Code, BlueJ

O Android Studio foi utilizado para desenvolver a aplicação Android, manter uma estrutura organizada, trabalhar com Java e XML e realizar testes utilizando um emulador de telemóvel.

O VS Code foi utilizado para desenvolver o site promocional e painel de administração, trabalhar com HTML, CSS, JavaScript, PHP e SQL e as suas extensões permitem realizar testes de todo o projeto.

O BlueJ foi utilizado para desenvolver as operações na fase inicial do projeto, para desenvolver os modelos de dados e para testes também em fases iniciais. A sua interface em blocos torna as operações e relações de objetos serem mais intuitivas.

Ferramentas de Teste - Postman, Ngrok, Mailtrap

O Postman foi utilizado para simular a ligação entre a API e a aplicação Android assim sendo mais fácil trabalhar com rotas e permite que alguém a trabalhar apenas na API consiga fazer todos os testes necessários.

O Ngrok foi utilizado para criar um túnel do servidor local na web o que torna possível fazer a ligação da API com a app Android.

O Mailtrap foi utilizado para os testes de verificação de email quando um utilizador faz o seu registo.

Servidor Local - Laragon

O Laragon foi utilizado para criar um servidor local o que permite simular como o negócio funcionaria na web.

Base de Dados - MySQL

O MySQL foi utilizado para criar e gerir a base de dados do projeto, armazenando toda a informação necessária para o funcionamento do sistema.

Design e Produção Visual - Figma, Canva, Ibis Paint, PicsArt

O Figma foi utilizado para desenvolver os protótipos e mockups da aplicação em Android.

O Canva foi utilizado para desenvolver a apresentação e alguns elementos visuais.

O Ibis Paint e O PicsArt foram utilizados para desenvolver o design da maioria dos ícones, imagens e artstyle do projeto.

Gestão de Código - Git e GitHub

O Git e O Github foram utilizados para a gestão do projeto tornando possível guardar versões e partilhá-las entre os membros do projeto.

7.3 Cronograma

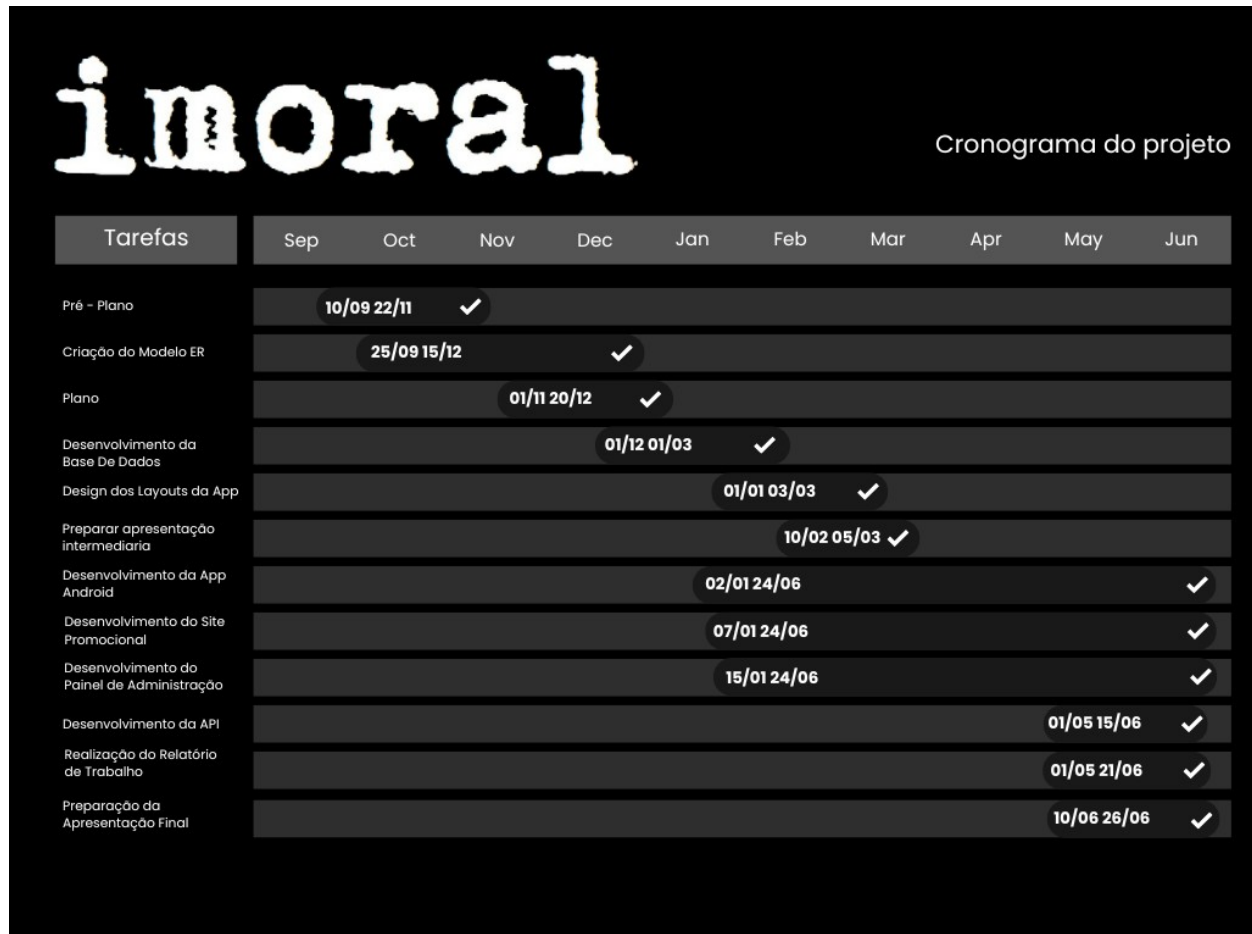


Fig.1- Cronograma imoral

Pré-Plano e Plano - 10/09 a 20/12: Anteriormente ao desenvolvimento do projeto, foi feito um Pré-Plano e um Plano, que descreve quais eram os objetivos, quais as motivações e quais as tecnologias iríamos utilizar no projeto.

Criação do Modelo ER - 25/09 a 15/12: O desenvolvimento de um Modelo ER foi essencial para a definição da estrutura base do projeto. Apesar de ter sido necessário alterar a estrutura diversas vezes, sem o Modelo ER não teríamos uma base.

Desenvolvimento da Base de Dados - 01/12 a 01/03: A Base de Dados foi desenvolvida de forma relativamente simples graças ao Modelo ER já criado, tendo sido igualmente alterada ao longo do projeto.

Design dos Layouts da APP - 01/01 a 03/03: O Design do projeto foi uma etapa muito importante já que uma das nossas preocupações era passar a nossa imagem pelo projeto e foi o que tornou o desenvolvimento da App Android muito mais simples.

Preparação da Apresentação Intermédia - 10/02 a 05/03: A apresentação intermédia foi muito importante para saber quais seriam os nossos próximos passos para o seguimento do desenvolvimento do projeto e deu-nos uma ideia do que deveríamos focar.

Desenvolvimento da App Android - 02/01 a 24/06: A App Android por ser a parte principal do projeto foi a mais complexa, foi aqui que algumas das funcionalidades foram priorizadas em a outras. A ligação com a API foi uma área nova em que tivemos dificuldades.

Desenvolvimento do Site Promocional - 07/01 a 24/06: Nesta etapa foi desenvolvido o site que teria como função apresentar o nosso projeto e foi o primeiro contacto que tivemos com HTML, CSS e JavaScript.

Desenvolvimento do Pannel de Administração - 15/01 a 24/06: O Pannel de Administração foi outra parte muito importante do projeto já que ele tem como função gerir e ajudar a tomar decisões do nosso negócio.

Desenvolvimento da API - 01/05 a 15/06: O desenvolvimento da API foi uma das partes mais importantes do projeto já que com ela o sistema passa a funcionar a um nível real e não estático. Foi também uma das partes que me gerou mais interesse ao longo do projeto.

Realização do Relatório de Trabalho - 01/05 a 21/06: O Relatório é a parte onde é recolhido tudo o que foi feito do projeto de forma a documentar os conteúdos que desenvolvemos.

Preparar a Apresentação Final - 10/06 a 26/06: A última etapa do projeto é a preparação para a apresentação final onde será apresentado o que foi feito no projeto e explicar o que representa para nós.

8. Desenvolvimento

8.1 Arquitetura do Sistema

O sistema é composto por quatro componentes principais, a aplicação Android, o site promocional, o painel de administração e a base de dados MySQL. A aplicação Android comunica com a base de dados utilizando uma API em PHP. O site promocional e o painel de administração fazem ligação direta com a base de dados utilizando também PHP mas sem passar pela API.

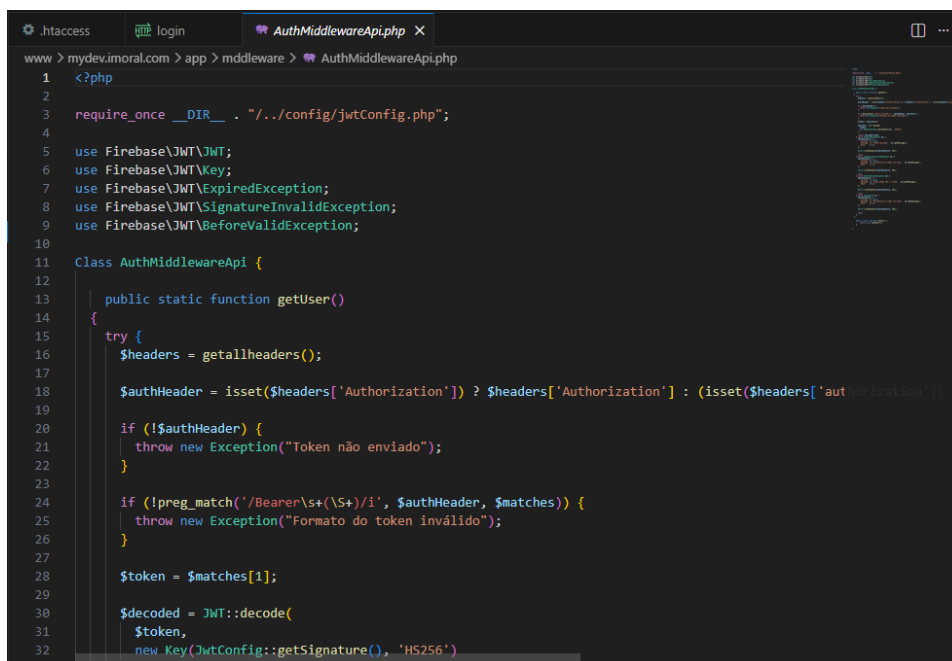
A **API** funciona como uma "ponte" que faz uma ligação entre diferentes sistemas que podem ter diferentes linguagens de programação, sendo responsável por transferir todos os dados desses sistemas de forma correta.

Middleware -

O **Middleware** é aquele que faz a segurança das rotas da API, aqui ele verifica se o token JWT é valido e apenas se for ele deixa o utilizador aceder as rotas.

Exemplo:

Esse é o codigo de verificação do middleware ele verifica se o token é valido ou invalido:



```
1 <?php
2
3 require_once __DIR__ . '/../config/jwtConfig.php';
4
5 use Firebase\JWT\JWT;
6 use Firebase\JWT\Key;
7 use Firebase\JWT\ExpiredException;
8 use Firebase\JWT\SignatureInvalidException;
9 use Firebase\JWT\BeforeValidException;
10
11 Class AuthMiddlewareApi {
12
13     public static function getUser()
14     {
15         try {
16             $headers = getallheaders();
17
18             $authHeader = isset($headers['Authorization']) ? $headers['Authorization'] : (isset($headers['authorization'])
19
20             if (!$authHeader) {
21                 throw new Exception("Token não enviado");
22             }
23
24             if (!preg_match('/Bearer\s*(\S+)/i', $authHeader, $matches)) {
25                 throw new Exception("Formato do token inválido");
26             }
27
28             $token = $matches[1];
29
30             $decoded = JWT::decode(
31                 $token,
32                 new Key(JwtConfig::getSignature(), 'HS256')
```

Fig.2- AuthMiddleware Visual Code

Aqui uma verificação bem sucedida um erro de verificação:

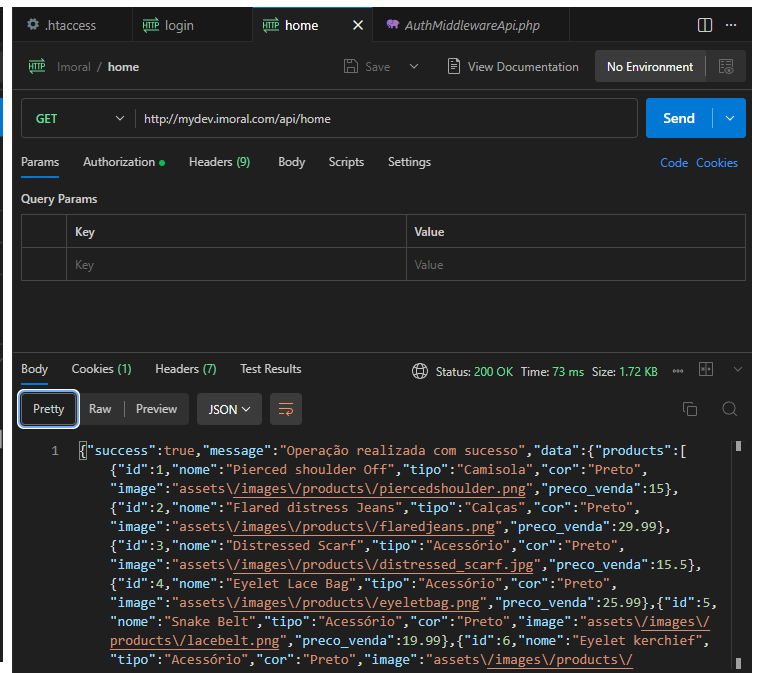
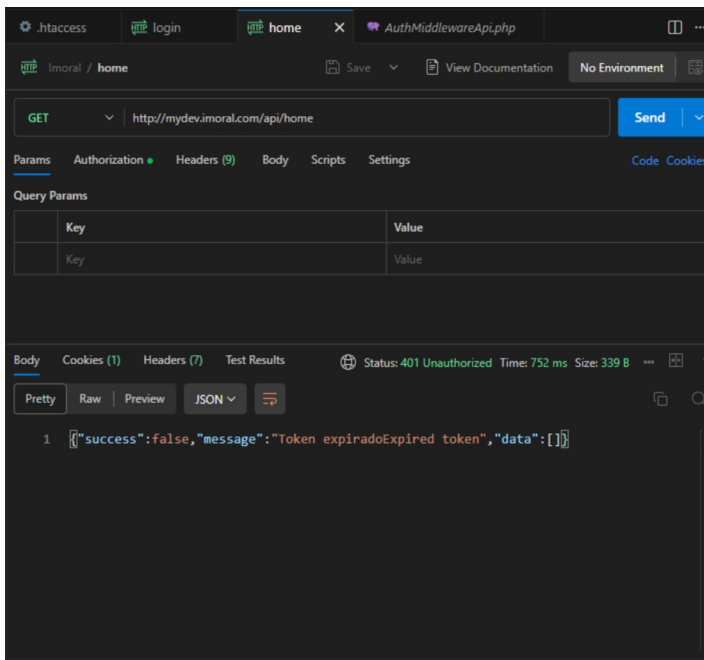


Fig.3 e Fig.4- Erro de Verificação e Verificação Bem Sucedida.

MVC (Model View Controller) -

Nosso projeto utiliza o MVC é uma forma de estrutura de sistema. MVC significa Model View Controller ou seja, o código divide-se em 3 partes:

Para demonstrar será utilizado como exemplo os utilizadores na API:

```
www > mydev.imoral.com > app > models > User.php
1  <?php
2
3  class User {
4
5      private int $id;
6      private string $username;
7      private string $email;
8      private ?string $image;
9      private ?string $telefone;
10     private string $password;
11     private string $morada;
12     private string $dt_nascimento;
13     private string $dt_criacao;
14     private ?string $pronomes;
15     private bool $is_admin;
16     private string $ultimo_login;
17     private bool $is_verified;
18     private ?string $verified_at;
19     private string $created_at;
20     private string $updated_at;
21     private ?string $deleted_at;
22
23     public function __construct(
24         int $id = 0,
25         string $username = '',
26         string $email = '',
27         ?string $image = null,
28         ?string $telefone = null,
29         string $password = '',
30         string $morada = '',
31         string $dt_nascimento = '',
32         string $dt_criacao = '',
```

Fig.5- Model de Utilizadores

O **Model** é uma classe com os mesmos atributos da tabela da base de dados.

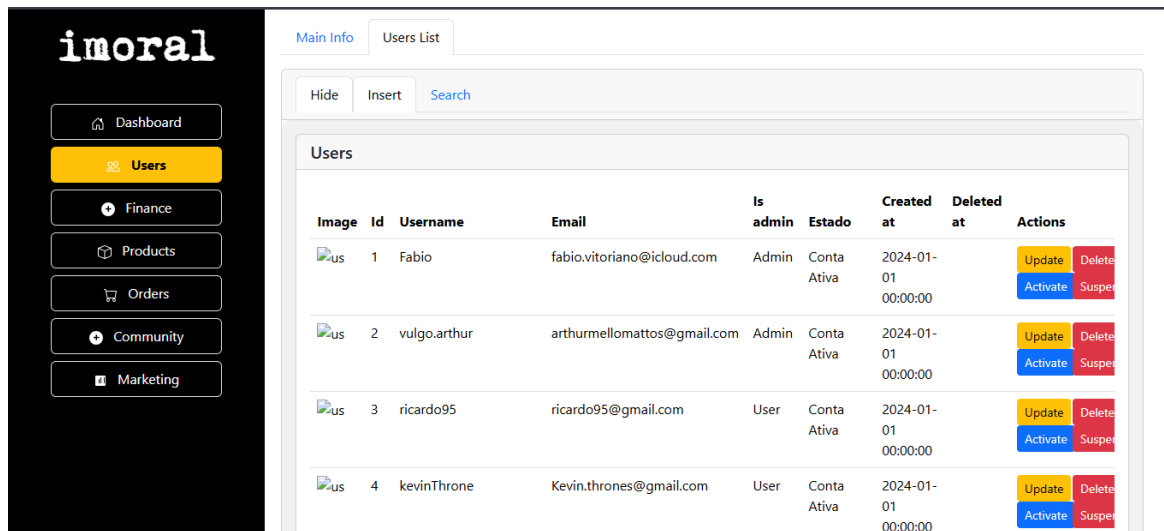


Fig.6- View Lista de Utilizadores

A **View** é a parte visual, ou seja o que o utilizador vê. Aqui temos a View da dashboard na secção que os utilizadores são chamados.

```

www > mydev.imoral.com > app > controllers > UserController.php
7 {
58
59 public function getAllUsers($userId) {
60     try {
61         $users = (new UserDAO())->arrayUsersDAO();
62         $emailsVerification = (new EmailVerificationDAO())->getVerificationsByUserId($userId);
63         $countUsers = (new UserDAO())->countUsers();
64         $countEmailsVerification = (new EmailVerificationDAO())->countVerifications();
65
66         $dataResponse = [
67             'success' => true,
68             'message' => "Operação realizada com sucesso",
69             'data' => [
70                 'users' => $users,
71                 'emails_verification' => $emailsVerification,
72                 'num_users' => $countUsers,
73                 'num_emails' => $countEmailsVerification
74             ]
75         ];
76
77         Utils::jsonResponse($dataResponse, 200);
78
79     } catch (Exception $e) {
80         $dataResponse = [
81             'success' => false,
82             'message' => $e->getMessage(),
83             'data' => []
84         ];
85
86         Utils::jsonResponse($dataResponse, 401);
87     }
88 }

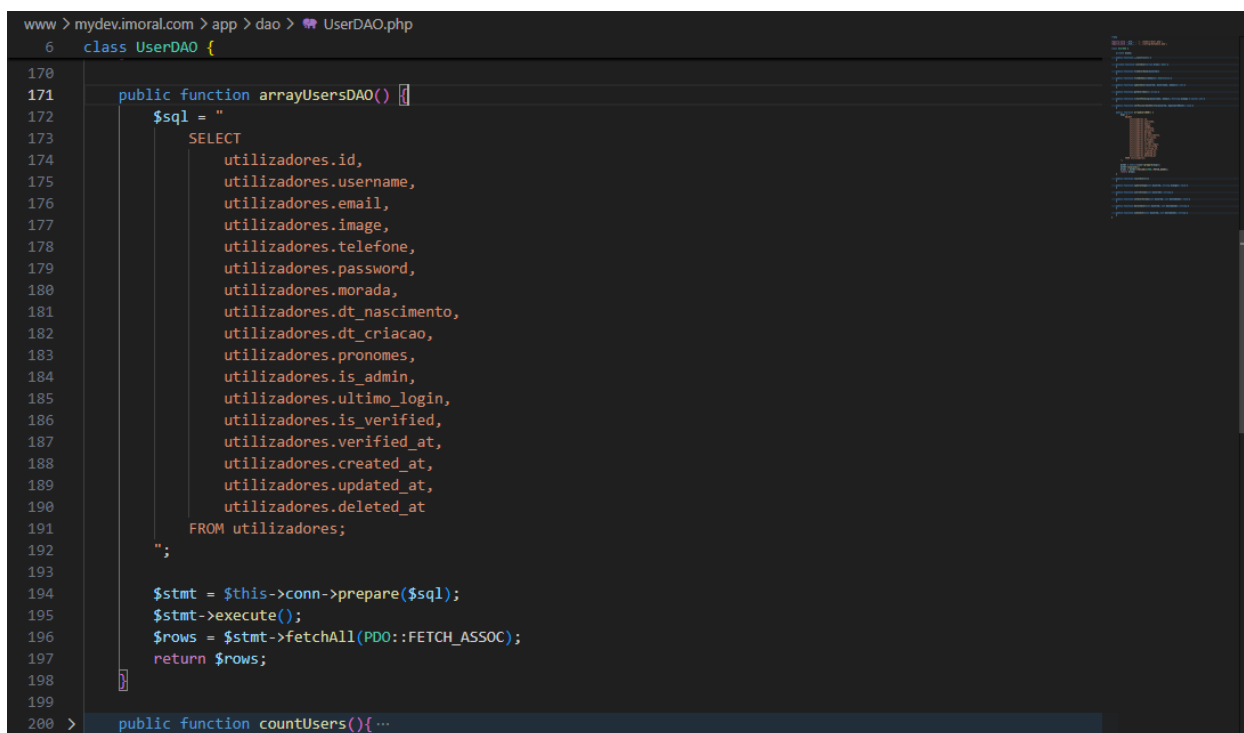
```

Fig.7– UserController getAllUsers

O **Controller** é um controlador, ele liga o Model a View, processa os pedidos e devolve o que foi pedido.

DAO (Data Access Object)-

No projeto também foram utilizados **DAO's** que significa Data Access Object é um padrão de organização, DAO's são classes que são responsáveis por fazer as operações das queries na base de dados. Isso faz com que as queries fiquem todas centralizadas numa classe das operações de uma tabela.



```
www > mydev.imoral.com > app > dao > UserDAO.php
6 class UserDAO {
170
171     public function arrayUsersDAO() {
172         $sql = "
173             SELECT
174                 utilizadores.id,
175                 utilizadores.username,
176                 utilizadores.email,
177                 utilizadores.image,
178                 utilizadores.telefone,
179                 utilizadores.password,
180                 utilizadores.morada,
181                 utilizadores.dt_nascimento,
182                 utilizadores.dt_criacao,
183                 utilizadores.pronomes,
184                 utilizadores.is_admin,
185                 utilizadores.ultimo_login,
186                 utilizadores.is_verified,
187                 utilizadores.verified_at,
188                 utilizadores.created_at,
189                 utilizadores.updated_at,
190                 utilizadores.deleted_at
191             FROM utilizadores;
192         ";
193
194         $stmt = $this->conn->prepare($sql);
195         $stmt->execute();
196         $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
197         return $rows;
198     }
199
200     public function countUsers(){...
```

Fig.8– UserDAO arrayUsersDAO.

8.2 Design da Interface

A imoral é um projeto que reflete muito a personalidade dos seus criadores tornando-o um projeto muito artístico o que faz ter um visual que reflete isso. Procurou-se com a imoral passar uma versão da realidade chocante que vivemos e queríamos trazer uma estética que é propositalmente "imoral". A cor predominante é preto o que faz com que todo o resto puxe o olhar e tenha um destaque grande. Também tem uma estética crua com a maioria das imagens em preto e branco forte.

App Android: Com a App Android procurou-se ter um visual chocante mas que mesmo assim intuitivo. Foram retiradas inspirações de designs de apps já existentes para os utilizadores se sentirem confortáveis.



Fig.9- Layouts do Figma da App Android

Site Promocional: No Site Promocional foi usado basicamente o mesmo padrão. Procurou-se novamente um visual chocante com uma interface intuitiva.

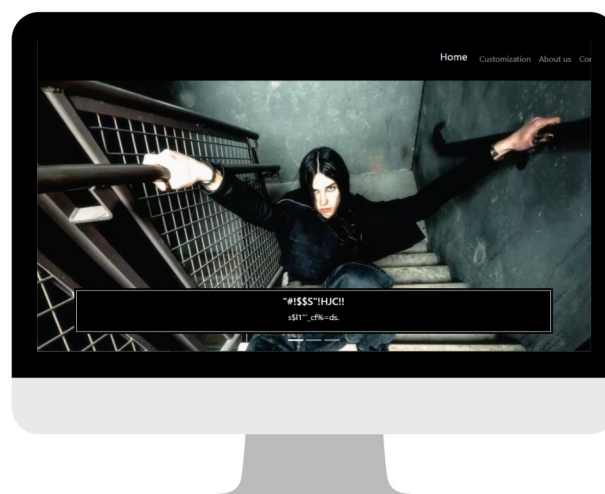


Fig.10- Design do Site Promocional

Painel de Administração: Já no Painel de Administração o objetivo foi ser o mais visualmente confortável e prático possível já que ele é apenas para fazer a gestão do negócio.

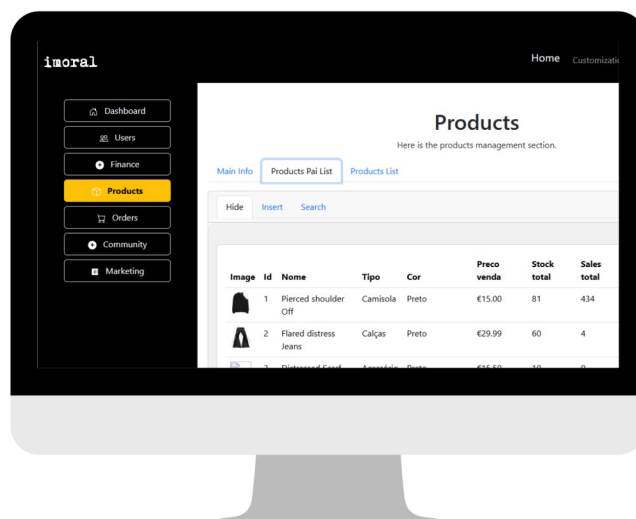


Fig.11- Design do Painel de Administração

8.3 Implementação

O desenvolvimento do projeto seguiu a ordem de como os conhecimentos foram adquiridos ao longo deste ano sendo primeiro a base de dados, site e painel, app e por fim a API. No entanto foram constantemente realizadas alterações em todas as partes do projeto durante todo o tempo do seu desenvolvimento.

Base de Dados -

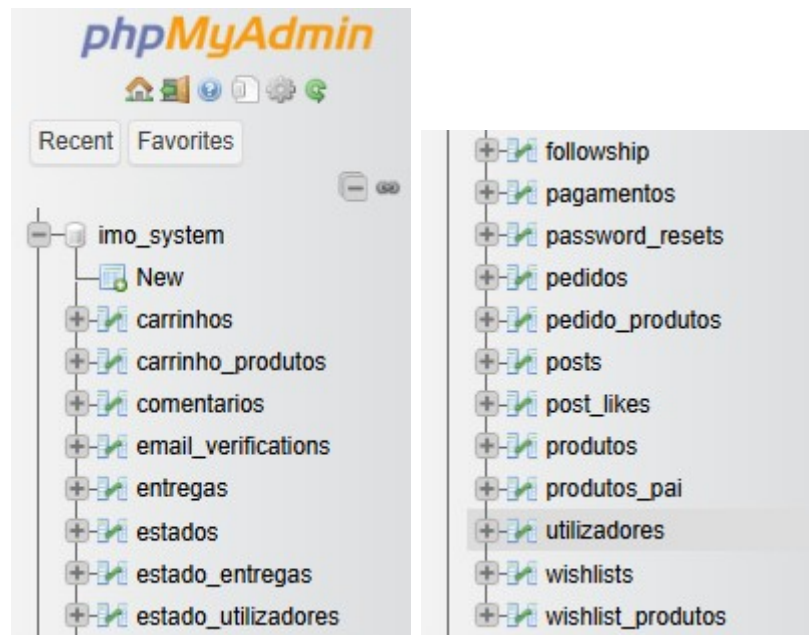


Fig.12 e Fig.13- Campos Base de Dados

A base de dados é composta por 20 tabelas que servem para gerir dois negócios distintos a loja e o espaço comunidade e podem ser divididos em quatro grupos principais: utilizadores e autenticação, espaço comunidade, loja e encomendas e pagamentos.

Conceito de Chaves Primarias(PK) e Chaves Estrangeiras(FK) -

Conceito de Chaves Primárias (PK) e Chaves Estrangeiras (FK) - Chaves Primárias são identificadores únicos que não podem se repetir, têm o papel de identificar um conjunto de dados específicos como o utilizador 1, 2, 3 ou 4, cada um com um conjunto de dados único. As Chaves Estrangeiras têm como papel referenciar outras tabelas para a comunicação entre tabelas. Chaves Estrangeiras são Chaves Primárias noutras tabelas.

Utilizadores e Autenticação -

utilizadores, email_verifications, password_resets e estado_utilizadores

Essas tabelas são responsáveis por armazenar os dados dos utilizadores e também tem o papel de atuar na autenticação e nas informações relativas ao estado da conta, se ela está suspensa ou não

Espaço Comunidade -

followship, posts, like_count e comentários

Essas tabelas são responsáveis por armazenar os dados do espaço comunidade armazenando os posts e os comentários, a followship, quem segue e quem é seguido por alguém, e os gostos, registrando quem reagiu a cada publicação.

Loja -

produtos_pai, produtos, wishlists, wishlists_produtos_pai, carrinhos e carrinho_produtos.

Essas tabelas são responsáveis por armazenar os dados da loja incluindo todos os produtos pai que contêm o nome, tamanho, cor e preço de venda e os produtos que contêm as restantes informações mais específicas, as wishlists dos utilizadores e os carrinhos dos utilizadores.

Encomendas e Pagamentos -

pedidos, pedido_produtos, entregas, pagamentos, estados e estado_entregas.

Essas tabelas são responsáveis por armazenar os dados das encomendas e pagamentos os pedidos que são os carrinhos após a compra ser efetuada, as entregas que são os pedidos quando enviados, os pagamentos que são os dados de pagamento dos utilizadores e os estados que definem em que fase se encontra cada encomenda e entrega, bem como o estado da conta dos utilizadores.

Modelo ER -

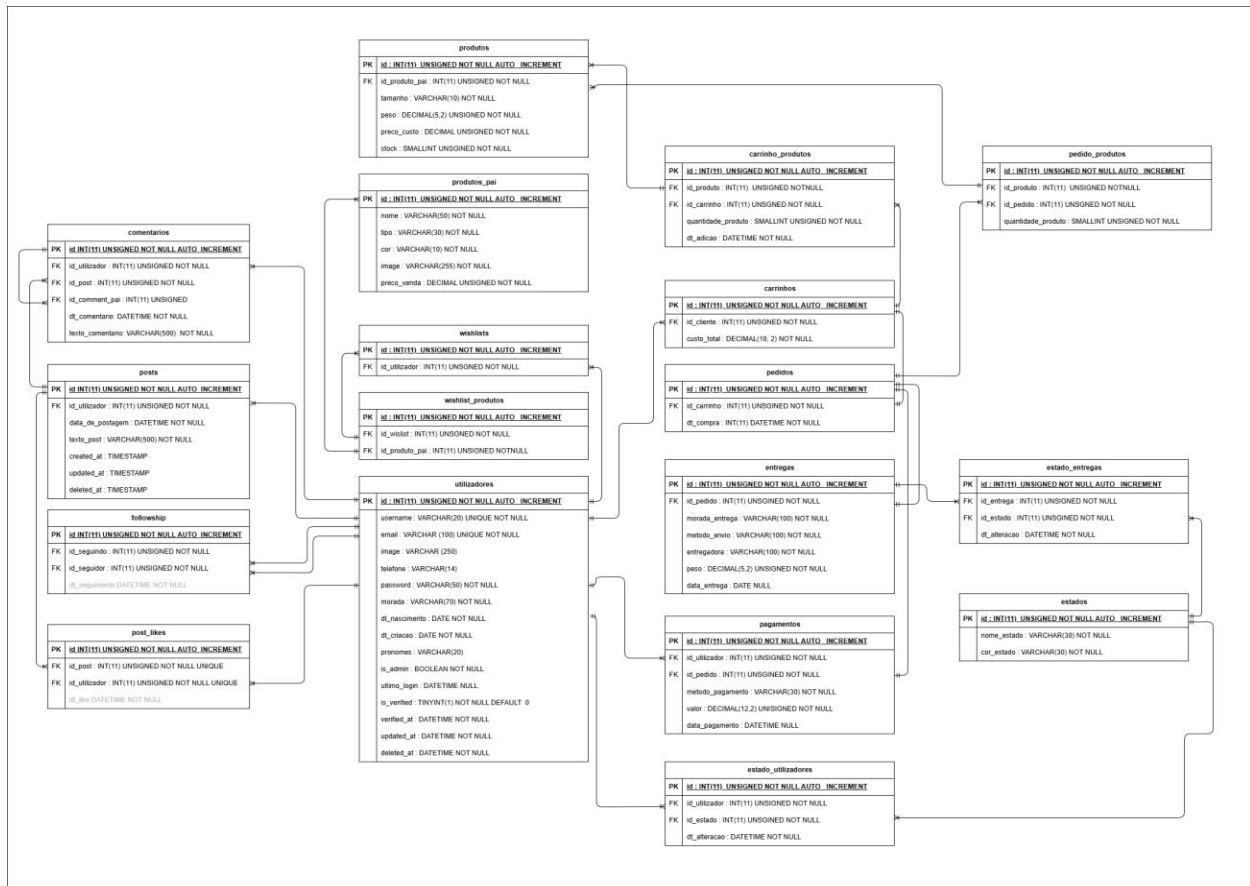


Fig.14- Modelo ER

Ligação da API com a Base de Dados: A ligação com a base de dados é feita com o ficheiro Database.php e ele funciona da seguinte maneira: Primeiro colocamos os atributos da base de dados sendo estes o host, o nome da base de dados e as informações de login, username e palavra-passe, depois vem o método connect() esse método faz um novo PDO, PDO é PHP Data Objects que é uma forma de fazer a ligação do PHP com a base de dados ele tem como função fazer a ligação com a base de dados, executar queries de forma segura e evitar ataques de SQL Injection, depois o método faz setAttribute que define o modo dos erros e faz o PDOException que é caso ocorra um erro de ligação.

```
www > mydev.imoral.com > app > config > Database.php
1  <?php
2
3  class Database
4  {
5      private $host = 'localhost';
6      private $db_name = 'imo_system';
7      private $username = 'root';
8      private $password = '';
9
10     public function connect()
11     {
12         $conn = null;
13
14         try {
15             $conn = new PDO(
16                 "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
17                 $this->username,
18                 $this->password
19             );
20
21             $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
22
23         } catch (PDOException $e) {
24             echo "Erro de conexão: " . $e->getMessage();
25         }
26
27         return $conn;
28     }
29 }
30
31
```

Fig.15- Ficheiro Database.php

API -

A API do projeto foi desenvolvida em PHP é do tipo API REST utilizando rotas para comunicar com a app Android. A autenticação é feita através de tokens JWT. A API suporta as operações GET, POST, PUT e DELETE. Durante todo o desenvolvimento da API os testes foram feitos no VS Code com a extensão do Postman, permitindo que o desenvolvimento da API seja independente da aplicação.

Login e Autenticação: Aqui está o método de autenticação no AuthController nesta primeira parte são obtidos os campos preenchidos, verifica se não estão vazios, procura se o utilizador existe pelo seu e-mail e verifica se a password é válida, depois coloca no data o id do utilizador a sua role.

```
232
233 public function loginApi() {
234     try {
235         $body = json_decode(file_get_contents('php://input'), true);
236
237         $email    = trim($body['email'] ?? $_POST['email'] ?? '');
238         $password = trim($body['password'] ?? $_POST['password'] ?? '');
239
240         // Se não houver email ou password, mostrar erro
241         // é preciso lançar exceção para o index.php apanhar e mostrar o erro via flash message
242         if (empty($email) || empty($password)) {
243             throw new Exception("Email e password são obrigatórios");
244         }
245
246         $user = (new UserDAO())->findByEmail($email);
247
248         if (!$user || !password_verify($password, $user->getPassword())) {
249             throw new Exception("Email ou password errados");
250         }
251
252         $data = [
253             'id' => $user->getId(),
254             'role' => $user->isAdmin()
255         ];
256     }
```

Fig.16- Primeira Parte do Código loginAPI.

De seguida obtém a configuração JWT, codifica com JWT e cria o dataResponse enviando os dados e dando a mensagem de sucesso, no caso de falhar ele manda a mensagem do erro que ocorreu pelo caminho.

```
257     $payload = jwtConfig::getConfig($data);
258
259     $jwt = JWT::encode($payload, jwtConfig::getSignature(), "HS256");
260
261     $dataResponse = [
262         'success' => true,
263         'message' => "Login efetuado com sucesso",
264         'data' => [
265             'jwt' => $jwt,
266             'user' => [
267                 'id' => $user->getId(),
268                 'is_admin' => $user->isAdmin(),
269                 'username' => $user->getUsername(),
270             ]
271         ]
272     ];
273
274     Utils::jsonResponse($dataResponse, 200);
275
276 } catch(Exception $e) {
277     $dataResponse = [
278         'success' => false,
279         'message' => $e->getMessage(),
280         'data' => []
281     ];
282
283     Utils::jsonResponse($dataResponse, 401);
284 }
285 }
```

Fig.17- Segunda Parte do Código loginAPI.

JWT Config: Aqui é onde é feita a configuração do JWT. Primeiro tem o método `getSignature()` que serve para obter a chave JWT que codifica os dados, depois vem o método `getConfig()` que é o método que passa a configuração JWT, `iat` é a data de criação do token, `exp` é a data de expiração do token que no caso é 1 hora e o `data` é onde os dados do utilizador são guardados.

```
www > mydev.imoral.com > app > config > JwtConfig.php
1  <?php
2
3  class JwtConfig {
4
5      public static function getSignature(): string {
6          $secret = $_ENV['JWT_SECRET'] ?? null;
7
8          if (!$secret) {
9              throw new \RuntimeException('JWT_SECRET não definido!');
10         }
11
12         return $secret;
13     }
14
15     public static function getConfig(mixed $data): array {
16         return [
17             'iat' => time(),
18             'exp' => time() + 3600,
19             'data' => $data
20         ];
21     }
22 }
23
24 ?>
25
```

Fig.18- Código do JwtConfig.

App Android -

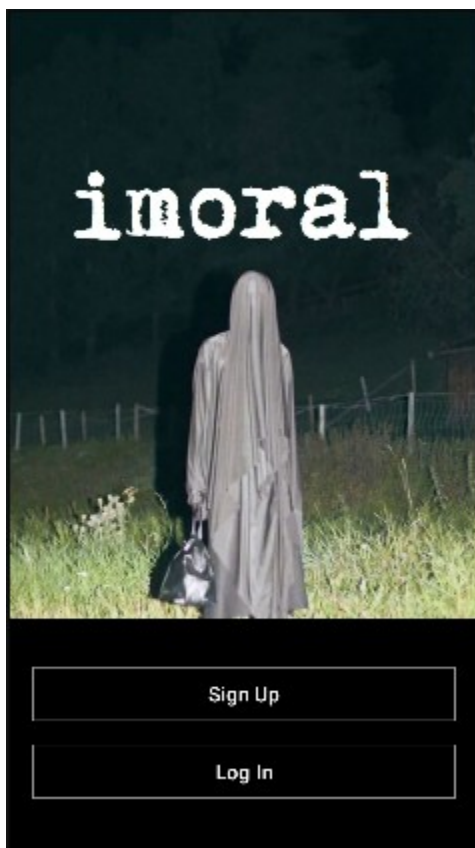


Fig.19- Entrada da App

A aplicação do projeto foi desenvolvida em Java, utilizando o Android Studio. Está organizada em Activities que são basicamente espaços independentes e cada Activity tem a sua view que é a parte visual da aplicação. A comunicação com a API é feita através de chamadas HTTP, sendo o Ngrok utilizado para fazer um "túnel" do servidor local durante o desenvolvimento. Tem uma estrutura de models que são classes que tem os mesmos atributos das tabelas SQL.

Quanto as funcionalidades implementadas temos:

Fazer Login e Signup -

Ver Home / Lista dos produtos principais disponíveis -

Ver e adicionar produtos a Wishlist -

Ver e adicionar produtos ao Carrinho -

Ver e Publicar posts no espaço comunidade -

Ver perfis de utilizadores -

Editar seu Perfil -

Ligação da App com a API: Mostrando com o login na App essa parte funciona da seguinte maneira: são obtidos os campos preenchidos, verifica se não estão vazios, de seguida cria o pedido HTTP com o OkHttpClient e o forma com o RequestBody e o Request.

```
1 usage
111 private void fazerLogin() {
112     String email = editTextEmail.getText().toString().trim();
113     String password = editTextPassword.getText().toString().trim();
114
115     if (email.isEmpty()) {
116         editTextEmail.setError("Email obrigatório");
117         return;
118     }
119
120     if (password.isEmpty()) {
121         editTextPassword.setError("Password obrigatória");
122         return;
123     }
124     OkHttpClient client = new OkHttpClient();
125
126     RequestBody formBody = new FormBody.Builder()
127         .add("email", email)
128         .add("password", password)
129         .build();
130
131     Request request = new Request.Builder()
132         .url(ApiConfig.LOGIN_URL)
133         .post(formBody)
134         .build();
135 }
```

Fig.20- Primeira Parte Login App

Após isso recebe a resposta no Callback se der erro ele devolve a mensagem de erro com o erro vindo da API e se for sucesso ele também dá um toast mas de sucesso e guarda o JWT nas SharedPreferences e por fim envia o utilizador para a MainActivity.

```
134         .build();
135
136     client.newCall(request).enqueue(new Callback() {
137         @Override
138         public void onFailure(@NonNull Call call, @NonNull IOException e) {
139             Toast.makeText(context: LoginActivity.this, text: "Erro: " + e.getMessage(), Toast.LENGTH_SHORT).show();
140         }
141
142         @Override
143         public void onResponse(@NonNull Call call, @NonNull Response response) throws IOException {
144             String responseBody = response.body() != null ? response.body().string() : "";
145             String statusCode = String.valueOf(response.code());
146             try {
147                 LoginResponse loginResponse = gson.fromJson(responseBody, LoginResponse.class);
148
149                 runOnUiThread() -> {
150                     Toast.makeText(context: LoginActivity.this, text: "Code: " + statusCode + "\n" + loginResponse.c
151
152                     if (response.isSuccessful() && loginResponse != null && loginResponse.isSuccess()) {
153                         SharedPreferences prefs = getSharedPreferences(name: "app_session", MODE_PRIVATE);
154                         SharedPreferences.Editor editor = prefs.edit();
155                         editor.putString("jwt", loginResponse.getData().getJwt());
156                         editor.putInt("user_id", loginResponse.getData().getUser().getId());
157                         editor.putString("username", loginResponse.getData().getUser().getUsername());
158                         editor.putString("email", loginResponse.getData().getUser().getEmail());
159                     }
160                 }
161             } catch (Exception e) {
162                 Toast.makeText(context: LoginActivity.this, text: "Erro: " + e.getMessage(), Toast.LENGTH_SHORT).show();
163             }
164         }
165     });
```

Fig.21- Segunda Parte Login App

Site Promocional e Painel de Administração -

Ambos o Site Promocional e o Painel de Administração foram desenvolvidos em HTML, CSS e Javascript, utilizando o VS Code.

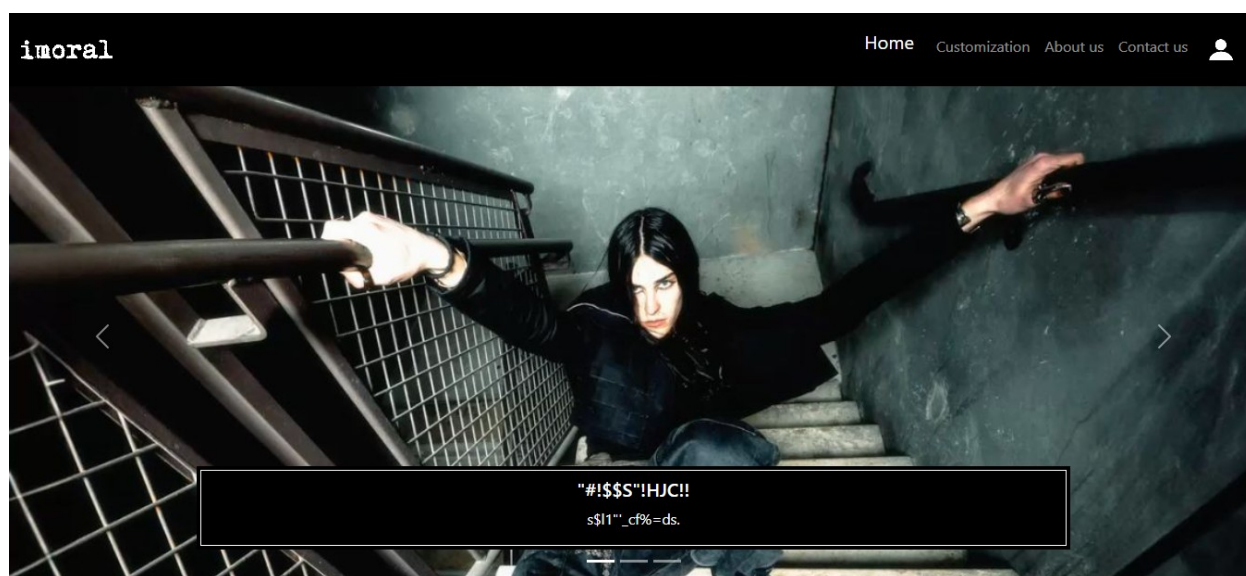


Fig.22- Site Promocional

O Site Promocional tem o papel de servir como página de entrada antes do acesso ao Painel de Administração, mas a sua função principal é apresentar a "imoral" para pessoas que não conhecem a marca.

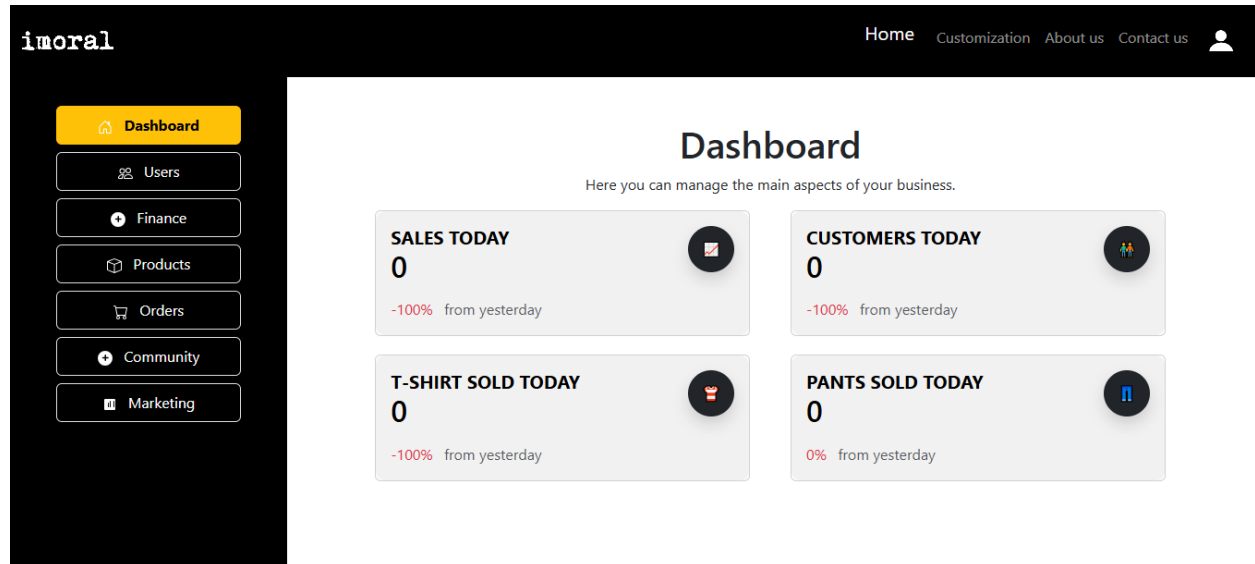


Fig.23- Paine de Administração

O Paine de Administração tem como principal função gerir o negócio e apresentar dados importantes para administração, tais como quantidade de utilizadores ativos e inativos, produtos adicionados, as vendas dos produtos, posts de utilizadores.

8.4 Testes e Validação

Durante o desenvolvimento do projeto para assegurar o funcionamento das funcionalidades ocorreram diversos testes, recorrendo a ferramentas como o emulador de telemóvel no Android Studio para a App Android, o Postman para a API e o browser para o site promocional e o paine de administração.

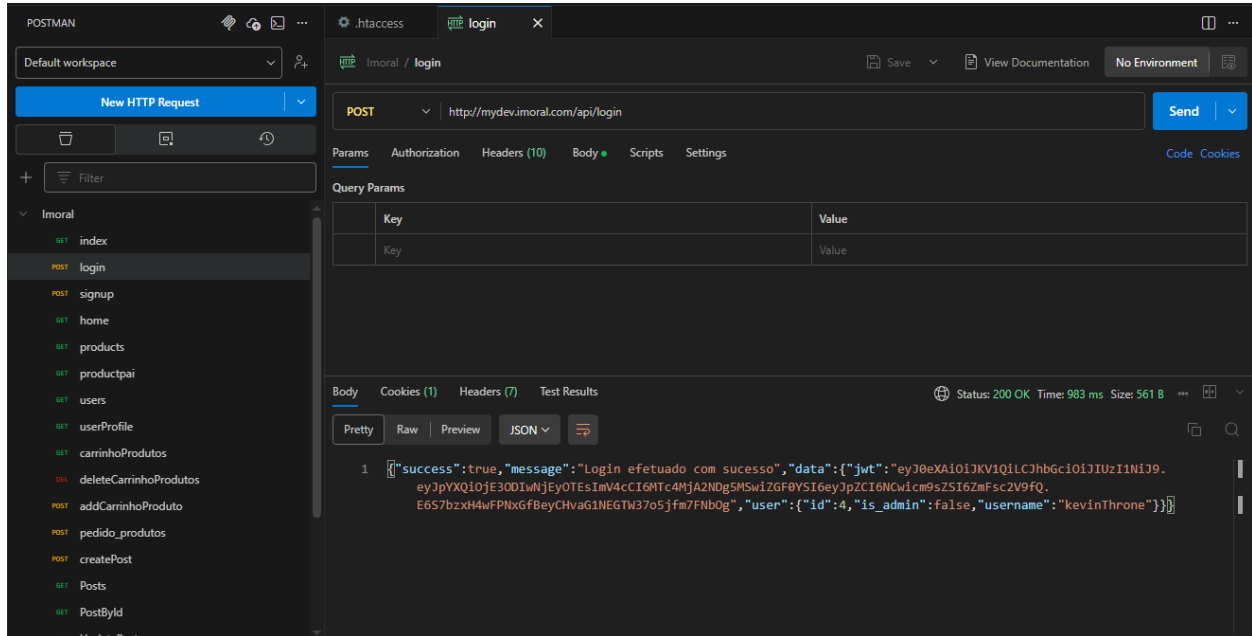


Fig.24- Login no Postman

Postman -

Como demonstrado anteriormente na secção de 8.1 Arquitetura do Sistema no Middleware, o Postman foi utilizado para testar as rotas protegidas da API, verificando o comportamento com tokens válidos e inválidos. Nele foi utilizado os tipos de request GET, POST, PUT e DELETE. Quanto ao token JWT é necessário fazer login pelo Postman e utilizar uma variável global do Postman ,depois enviar para o request no Header como um Bearer Token enquanto os dados são enviados no Body.

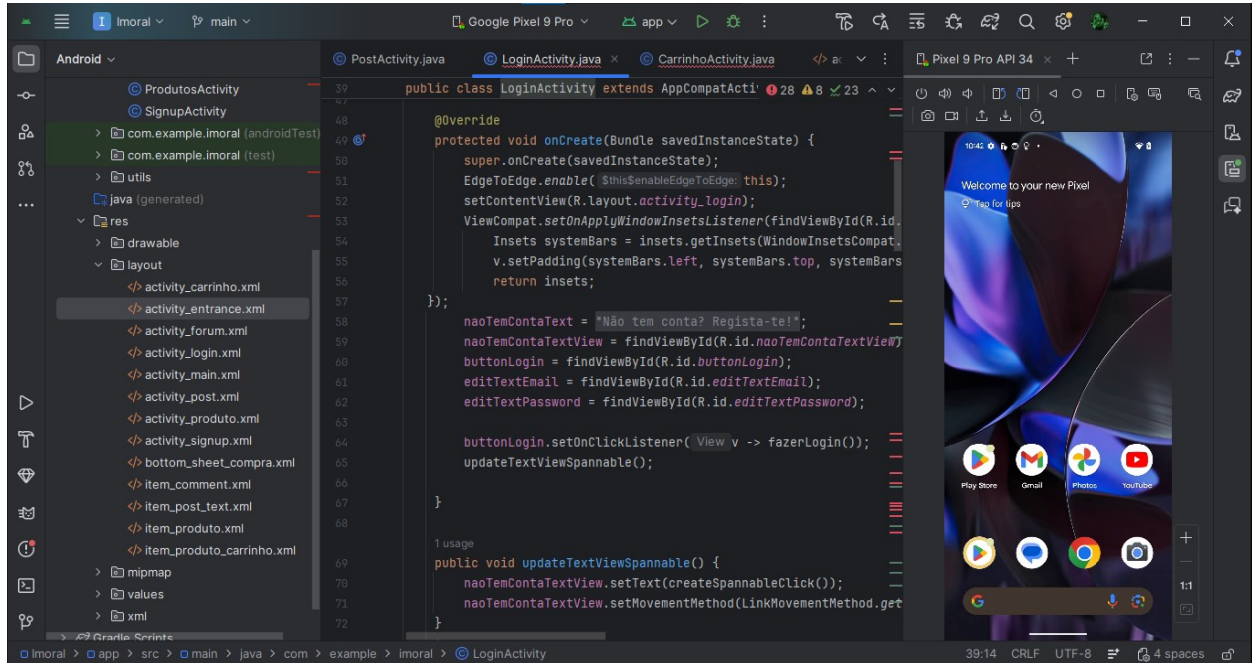


Fig.25- Emulador do Android Studio

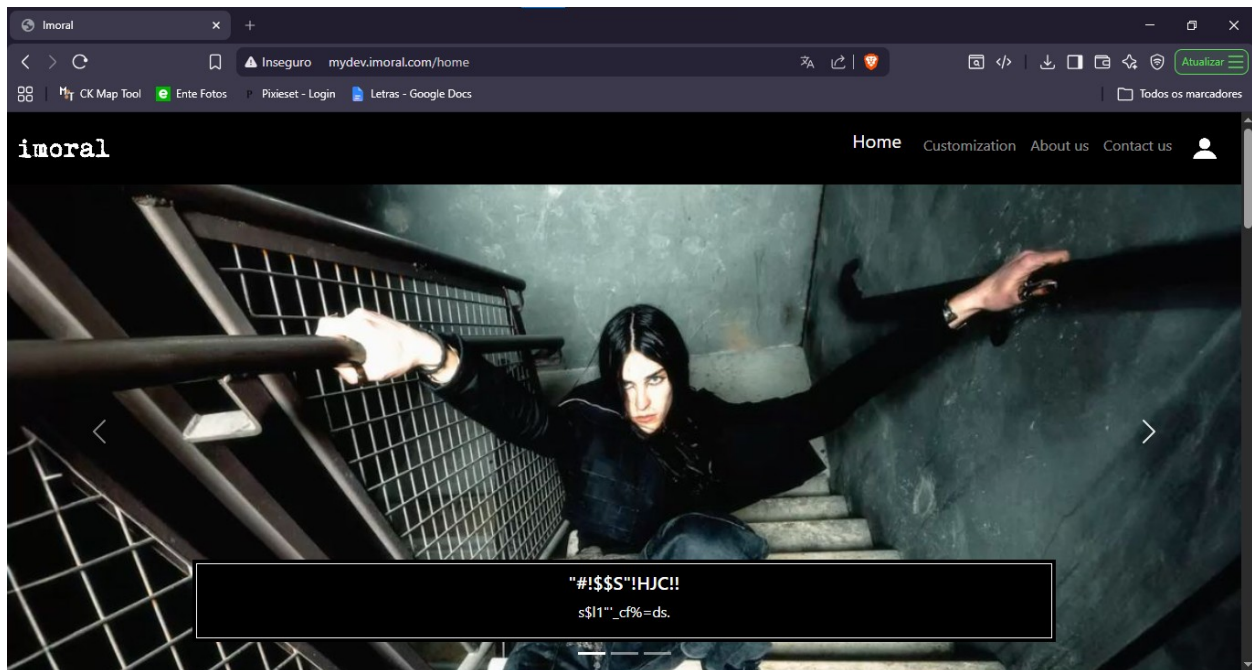


Fig.26- Site Promocional no Browser

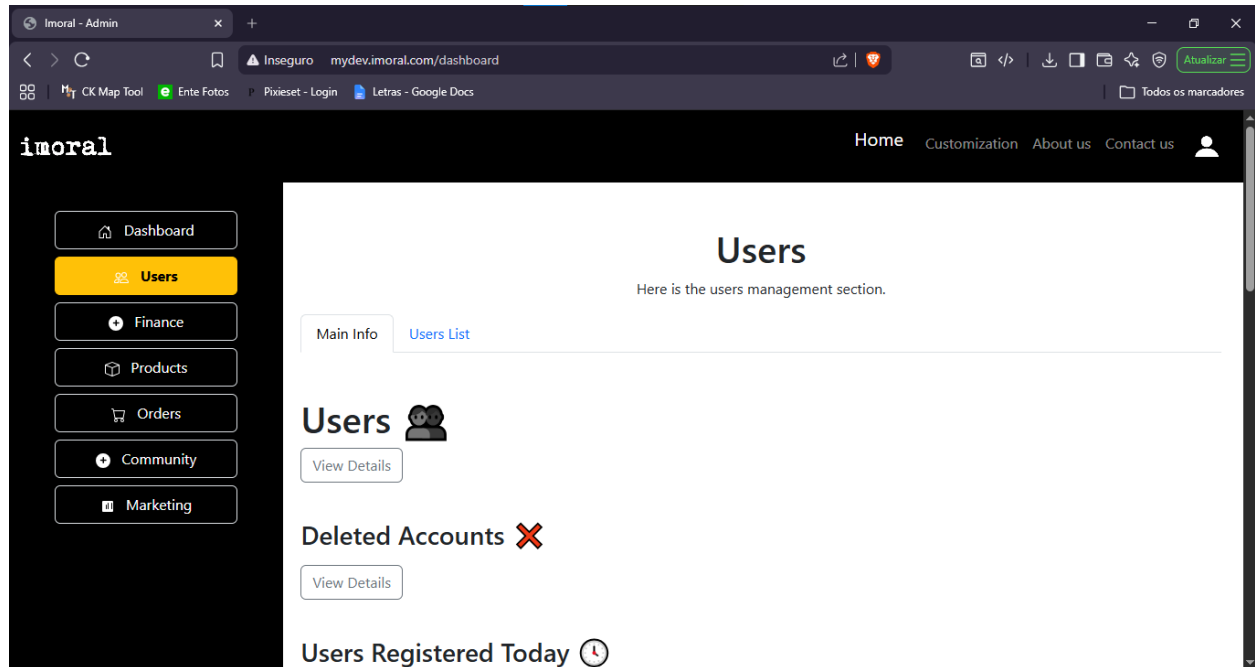


Fig.27- Painel de Administração no Browser

9. Resultados

9.1 Funcionalidades Desenvolvidas

Como resultados foi possível implementar várias funcionalidades no sistema tanto no site promocional, no painel de administração e na aplicação Android.

App Android

- Registo e Login
- Ver catálogo de produtos
- Adicionar produtos ao carrinho
- Adicionar produtos à wishlist
- Publicar posts no espaço comunidade
- Gostar e comentar posts
- Seguir outros utilizadores
- Ver perfis de utilizadores

Site Promocional

- Registo e Login para acesso ao Painel de Administração
- Apresentação da marca para o público

Painel de Administração

- Gestão de utilizadores, produtos e posts
- Moderação da comunidade

9.2 Benefícios Esperados

A imoral procura contribuir para uma mudança no meio cultural, levar as pessoas a expressarem-se é o nosso objetivo principal, ou seja procura que o seu benefício seja a expressão pessoal e o desenvolvimento pessoal dos nossos utilizadores. Mais especificamente esses são:

- Que os nossos utilizadores tenham consciência de quem são e do que eles próprios querem representar.
- Que marcas independentes tenham uma base para poder crescer como um coletivo.
- Que a comunidade alternativa ganhe mais força pelo que é e não pelo seu estereótipo.
- E que a consciência de expressão possa combater a padronização crescente.

9.3 Limitações

Apesar do objetivo de loja ter sido quase concluído tivemos limitações claras muito por causa da complexidade do projeto. Desenvolver uma loja completa que permita personalização e tenha um espaço comunidade demanda tempo e recursos alguns desses que não tivemos. Também por ser o nosso primeiro projeto surgiram algumas dificuldades . As principais limitações do nosso sistema são:

- App apenas disponível para Android.
- Não é possível selecionar produtos específicos para a compra dentro do carrinho
- Compras reais não podem ser efetuadas
- Personalização de produtos ainda não implementada
- O espaço comunidade ainda está em fase inicial não podendo ser realizadas funções essenciais

10. Conclusão

10.1 Síntese do Projeto

A imoral representa os seus criadores e o contexto que eles vivem, então a criação dela vem da nossa necessidade de ela existir. Conhecendo o cenário underground português verifica-se que faltam nele estrutura e abertura. Assim, decidiu-se criar uma loja de roupas e um espaço comunidade. Para isso foi desenvolvido uma App Android, um Site Promocional, um Painel de Administração e uma base de dados com uma API interligando todo o sistema. Por mais que os objetivos não tenham sido todos concluídos já existe uma boa base para seguir com o projeto no futuro.

Foram atingidos os objetivos de: desenvolver todos os seguintes espaços, a App Android, o Site Promocional, o Painel de Administração, a API, a base de dados

10.2 Reflexão Crítica

Com este projeto aprendi a desenvolver um sistema que abrange várias áreas, App, Site e Base de Dados e se conectam entre si com a API. Além disso melhorei muito as minhas capacidades de resolver e identificar problemas sejam de funcionamento ou estruturais. A nível pessoal houve uma melhoria em ultrapassar dificuldades em equipa e também como me expressar artisticamente em diferentes áreas.

Os maiores desafios foram quanto a estrutura do projeto tivemos que alterá-la diversas vezes, por ser o nosso primeiro projeto e por não sabermos planejar um sistema completo acabamos com estes problemas. Caso fosse começar do zero o planeamento e fazer uma estrutura melhor analisar possíveis erros e problemas futuros seria essencial.

Para mim este projeto representa um projeto colaborativo entre mim e o Fábio Vitoriano e representa como nos podemos expressarmos em conjunto.

10.3 Melhorias Futuras

Algumas melhorias futuras são essenciais para o projeto essas são:

- Concluir a simulação de compra na app
- Implementar a personalização de produtos
- Desenvolver o espaço comunidade completamente
- Permitir seleccionar produtos específicos no carrinho
- Expandir para iOS
- Alojjar o sistema online em vez de local

Além disso também seria bom rever a estrutura do projeto para evitar mais problemas estruturais.

11. Referências

W3Schools - site de consulta para HTML, CSS, JavaScript, PHP e SQL. Disponível em:
<https://www.w3schools.com>

YouTube - vídeos tutoriais sobre desenvolvimento Android, API REST em PHP e MySQL.